# Mobil Uygulama Testlerinde Modern Yaklaşımlar

## Cihaz Fragmantasyonundan Mağaza Onay Süreçlerine Uçtan Uca Kalite Güvencesi

Sunan: [İsminiz]

# Gündem ve Yol Haritası

Mobil test süreçlerini 4 ana eksende inceleyeceğiz.

## 1. Ortam ve Karmaşıklık

- Mobil Ekosistemin Doğası
- Test Piramidi
- Gerçek Cihaz vs. Simülatör

## 2. Senaryo Bazlı Testler

- Yaşam Döngüsü (Lifecycle)
- Kesinti (Interrupt) Testleri
- Ağ ve Bağlantı Senaryoları

## 3. Performans ve Güvenlik

- Donanım ve Kaynak Tüketimi
- Kullanıcı Deneyimi (UX)
- Veri Güvenliği

## 4. Süreç ve Dağıtım

- Mağaza İnceleme Süreçleri
- Vaka Analizleri
- Otomasyon ve CI/CD

# Mobil Ekosistemin Doğası

**Grup 6: Yazılım Karmaşıklığı'na Atıfla**

Mobil cihazlar, donanıma sıkı sıkıya bağlı bir entropi yaratır.

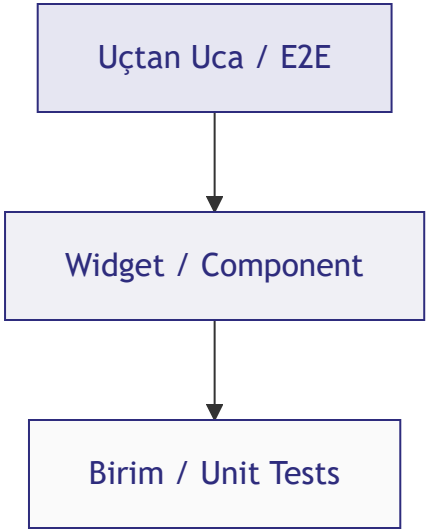
- OS Fragmantasyonu: Android pazarındaki binlerce farklı cihaz vs. iOS'in sürümler arası farklı yapısı.
- Ekran ve Çözünürlük: Farklı en-boy oranları ve piksel yoğunlukları.
- Donanım: Farklı işlemci mimarileri, GPU kapasiteleri ve sensörler.

## Karmaşıklık Yönetimi

Cihaz çeşitliliğinin yarattığı test kombinasyonları eksponansiyel olarak artar. Bu durum, Risk Tabanlı Test (RBT) stratejilerini zorunlu kılar.

# Test Piramidi ve Mobil Uygulamalar

Modern mobil mimaride testlerin katmanlı yapısı.




**Birim Testler (Unit):** İş mantığının ve API isteklerinin dışa bağımlı olmadan doğrulanması.

**Widget Testleri:** Arayüz bileşenlerinin (Örn: Flutter) bağımsız çalışabilirliği.

**Uçtan Uca (E2E):** Appium gibi araçlarla gerçek kullanıcı akışlarının simülasyonu.

# Gerçek Cihaz mı, Emülatör mü?

Test ortamı seçimi projenin bütçesini ve hızını doğrudan etkiler.

## 💻 Emülatör / Simülatör

- Avantaj: Hızlı kurulur, CI/CD'ye kolay entegre olur.
- Dezavantaj: Gerçek işlemci mimarisini yansıtmaz. Sensör testlerinde yetersizdir.
- Kullanım: UI testleri ve geliştirme.

## 📱 Gerçek Cihaz

- Avantaj: Kesin sonuç verir. Batarya, ısınma, bellek yönetimi ölçülebilir.
- Dezavantaj: Temini pahalıdır, bakımı zordur.
- Kullanım: Performans testleri ve canlı öncesi onay.

**Strateji: Hibrit Yaklaşım (Kritik senaryolar cihazda, diğerleri emülatörde)**

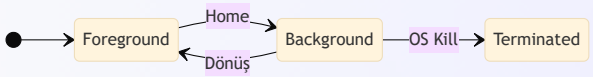
# Yaşam Döngüsü (Lifecycle)

Uygulama durumu işletim sistemi tarafından değiştirilir.

- Durum Yönetimi (State): Uygulama arka plana atıldığında veriler kayboluyor mu?
- Sistem Müdahalesi (OS Kill): İşletim sistemi RAM'i boşaltmak için uygulamayı acımasızca kapatır.
- Açılış Performansı:
  - *Cold Start:* Sıfırdan açılış.
  - *Warm Start:* Arkaplandan dönüş.

### Model Tabanlı Yaklaşım

Durumlar bir state machine olarak modellenir.



# Kesinti (Interrupt) Testleri

Mobil cihazlar "iletişim" aracıdır, testler buna göre yapılmalıdır.

Bir kullanıcı kritik bir işlem yaparken dışarıdan gelen bir kesintinin uygulamayı çökertmemesi gerekir.

## Yaygın Kesintiler

- Gelen GSM araması veya SMS
- WhatsApp / Anlık Bildirimler
- Düşük Pil Uyarısı

## Beklenen Davranış

- İşlem beklemeye (pause) alınmalı.
- Uygulama crash olmamalı.
- Kesinti bitince işlem devam etmeli.

# Ağ ve Bağlantı Senaryoları

Bağlantı her zaman sabit ve mükemmel değildir.

- Bant Genişliği Geçişleri: Kullanıcı asansöre bindiğinde 5G'den Edge'e düşen bağlantıda uygulama donuyor mu?
- Ağ Değişimi (Handover): Evden çıkarken Wi-Fi'dan hücresel veriye geçişte oturum (Session) kopuyor mu?
- Çevrimdışı (Offline) Mod: Uygulama internet yokken de temel işlevlerini yerine getirebilmeli.

Mühendislik Çözümü: Çevrimdışı yapılan işlemlerin, cihaz internete bağlandığı anda arka planda veritabanı ile otomatik senkronize edilmesinin test edilmesi.

# Exploratory ve Monkey Testing

Senaryosuz testin gücü: Otomasyonun göremediğini insan bulur.

## Exploratory Testing (Keşifsel)

Önceden yazılmış senaryo olmadan, uzman tarafından özgürce kullanılarak hata aranması.

- Ne Zaman: Yeni özellik geldiğinde, otomasyon yokken.
- Önemi: Kullanıcılar senaryoya uymaz; beklenmedik jestler ancak bu yöntemle yakalanır.

## Monkey Testing (Rastgelelik)

Android'in aracıyla uygulamaya rastgele dokunma olayları gönderilerek crash tespiti yapılır.

```
adb shell monkey -p com.app -v 10000
```

- Avantaj: Düşünülemeyen edge case'leri yakalar.
- Dezavantaj: Hatanın nedenini tekrar üretmek zordur.

# Performans ve Kaynak Tüketimi

Uygulamanın çalışması yetmez, cihazı yormamalıdır.

01

## Batarya Tüketimi

Arka planda çalışan servislerin (özellikle GPS ve Bluetooth) pili tüketme hızı.

02

## Bellek Sızıntısı

Uygulama açık kaldıkça RAM tüketiminin artması, çöp toplayıcının çalışamadığı durumlar.

03

## CPU ve Isınma

Aşırı işlemci yükünde cihazın frekans düşürmesi sonucu oluşan yavaşlamalar.

# Güvenlik Testleri

Taşınabilir cihazlar fiziksel ve dijital hırsızlığa açıktır.

## Veri Depolama

Hassas veriler asla düz metin kaydedilmemeli. iOS'te Keychain, Android'de Encrypted SharedPreferences kullanılmalı.

## Tersine Mühendislik

Kötü niyetli kişilerin APK/IPA kodlarını okumasını zorlaştırmak için kod karartma (Obfuscation) yapılmalı.

## API ve Trafik Güvenliği

Sunucu ile istemci arasındaki iletişimin araya girme saldırılarına (MitM) karşı SSL Pinning mimarisi ile korunması.

# Mağaza İnceleme (Store Review) Süreçleri

Kodun hatasız olması, uygulamanın yayınlanacağı anlamına gelmez.

- Apple ve Google Kriterleri: Mağazaların katı tasarım (Human Interface), gizlilik ve veri kurallarına uyumluluk kontrolü.
- Yasal ve Teknik Belgeler: Özellikle finans veya devlet uygulamalarında, mağazaya uygulamanın resmi bir kuruma ait olduğunu kanıtlayan "Authorization Letter" (Yetki Belgesi) sunulması zorunluluğu.
- Yaygın Red (Reject) Sebepleri:
  - Eksik meta veri (yanlış ekran görüntüleri).
  - Kullanıcıdan gereksiz donanım izni istenmesi.
  - İnceleme ekibi test ederken yaşanan herhangi bir "Crash".

# Lokalizasyon ve Uyumluluk

**Uygulama çalışıyor olabilir — ama doğru dilde ve formatta mı?**

## 1. Karakter Seti

- Türkçe özel karakterlerin (ş, ğ, ü) veritabanına doğru okunması.
- Arapça/İbranice (RTL) dillerde arayüzün ters dönmesi.

## 2. Format Farklılıkları

- Tarih: 25.04.2026 (TR) vs 04/25/2026 (US).
- Para: ₺1.250,99 vs $1,250.99
- Ödeme ekranlarında çökme yaratır.

## 3. Ekran Uyumu

- Almanca gibi dillerde buton taşması.
- Sistem dilini değiştirerek manuel test zorunludur.

Kalite Sorunu: Hedef pazarda yerelleştirilmemiş bir uygulama kullanıcı kaybeder.

# Vaka Analizleri

Gerçek dünya projelerindeki zorluklar.

## Senaryo 1: Belediye Uygulaması

Durum: E-devlet entegrasyonu barındıran hizmet uygulaması.

Zorluk: Apple inceleme ekibinin (Amerika) Türkiye sistemine girememesi. Özel hesaplar ve Demo Video ile aşıldı.

## Senaryo 2: Kargo Takip

Durum: Kuryenin konumunu ileten uygulama.

Zorluk: Kurye ekranı kilitlediğinde uygulamanın uyku moduna geçerek GPS'i kapatması.

Çözüm: "Foreground Service" yaşam döngüsü optimize edildi.

# Canlı Demo: Appium Login

## Mühendislik Pratikte Nasıl Görünür?

test_login.py

```python
from appium import webdriver

def test_login(driver):
    # Elementleri Bul
    user = driver.find_element_by_id("usr")
    btn = driver.find_element_by_id("btn")

    # Aksiyon
    user.send_keys("admin")
    btn.click()

    # Doğrula
    assert driver.find_element_by_id("ok")
```

Terminal • Running

```
$ pytest test_login.py

[Appium] Welcome
INFO: Finding element...
INFO: Sending keys...
INFO: Clicking button...
WAIT: Waiting (150ms)
=== 1 passed in 4.1s ===
```

✅ BAŞARILI

# Bulut Cihaz Çiftlikleri (Device Farms)

Gerçek cihaz testlerini ofisten buluta taşıyan mimari.

Kendi ofisinizde yüzlerce telefonu barındırmak yerine, kiralama modeliyle AWS veya Google veri merkezlerindeki gerçek telefonlara bağlanma sistemidir.

- Platformlar: AWS Device Farm, Firebase Test Lab, BrowserStack.
- Otomasyon Entegrasyonu: Appium scriptlerini buluta yükleyip tek tıkla 50 farklı cihazda paralel çalıştırabilirsiniz.
- Maliyet Analizi: Cihazların eskimemesi, batarya şişme riskinin olmaması ve 7/24 erişilebilirlik operasyonel maliyetleri ciddi oranda düşürür.

# Test Raporlama ve Metrikler

**Test ettik — peki sonucu nasıl ölçüyoruz?**

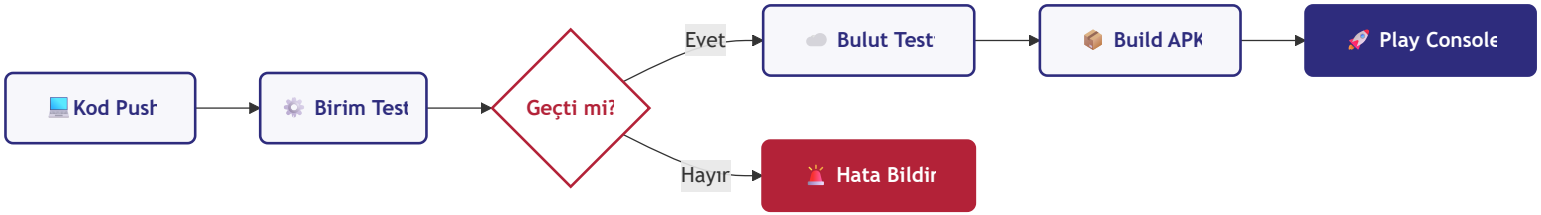
| | | | |
|---|---|---|---|
| **Crash Rate:** <%1 | **ANR Rate:** <%0.47 | **Coverage:** %70+ | **MTTD:** dakika |

| Araç | Ne İşe Yarar |
|---|---|
| **Firebase Crashlytics** | Gerçek kullanıcı crash'lerini anlık raporlar. |
| **Sentry** | Hem hata hem performans izleme, stack trace verir. |
| **Allure Report** | Appium sonuçlarını görsel rapora dönüştürür. |
| **Android Vitals** | Play Console içinde ANR ve crash dashboard'u. |

> **Özet:** Raporlama olmadan test, veri olmadan karar vermektir. CI/CD entegrasyonlu raporlar hata trendini izlemeyi sağlar.

# Sürekli Entegrasyon (CI/CD)

Kodun yazımından mağazaya gidişine kadarki üretim bandı.




> Her "Commit" sonrası insan eli değmeden otomatik testlerin koşturulması ve test ekibine Beta dağıtım yapılması sürecin kalbidir.

# Referanslar ve Kaynakça

Akademik ve Endüstriyel Kaynaklar:


1. ISTQB® - *Mobile Application Testing Foundation Level*.
2. Apple Developer - *App Store Review Guidelines*.
3. Google Play - *Developer Policy Center*.
4. Ders Notları: "Yazılım Test Mühendisliği" (Risk ve Karmaşıklık).
5. Appium Docs (appium.io).

# Teşekkürler

## Sorularınız & Tartışma

Sunumu PDF Olarak İndir

**Bu sunum GitHub Actions ve Slidev altyapısı kullanılarak kod tabanlı (as-code) hazırlanmıştır.**